Network Traffic Analysis Using Wireshark

Practical Portfolio Report

Prepared: 19 October 2026

Prepared by: ***A.W.P.S.D.Sandamali***

Project Results at a Glance

Practical	Observed result	Evidence status
01 — ICMP/DNS	Echo Request/Reply; DNS Query/Response visible	Capture saved
02 — TCP handshake	SYN, SYN-ACK, ACK observed in stream 10	Capture saved
03 — DHCP	Request and ACK; matching Transaction ID	Observed; no full DORA claim
04 — TCP/UDP	TCP 443→50698 PSH, ACK; UDP 51957→53, length 46	Screenshots and capture saved
05 — Retransmissions	67 filter matches; stream 1 identified	Field 50644 not fully confirmed
06 — Statistics	TCP 2,461 / 51,628 bytes; UDP 248 / 1,984 bytes	Corrected unfiltered values

Executive Summary

This project uses Wireshark to inspect captured network traffic and practice interpreting common network protocols. Six practical activities were completed or documented: ICMP/DNS inspection, TCP three-way handshake analysis, DHCP packet inspection, TCP/UDP packet analysis, TCP retransmission filtering, and protocol statistics.

The report records only observations provided during the practical work. Where a detail was not confirmed, it is explicitly identified as unverified rather than inferred.

Objectives

- Capture and inspect network packets using Wireshark.
- Recognize ICMP request/reply traffic and DNS query/response traffic.
- Identify the SYN, SYN-ACK, and ACK stages of a TCP three-way handshake.
- Inspect observed DHCP Request and ACK packets.
- Compare selected TCP and UDP packet header fields.
- Apply Wireshark's TCP retransmission analysis filter.

- Use Protocol Hierarchy and Conversations statistics to summarize captured traffic.

Tools and Environment

- Operating system: Windows 11.
- Packet analysis tool: Wireshark.
- Packet capture support: Npcap was installed/used in the capture environment.
- Network interface used for the practical captures: Wi-Fi.
- Only these environment details are included because they were established during the project workflow.

Methodology

Wireshark was used to capture traffic on the Wi-Fi interface. Display filters were applied to focus on ICMP, DNS, TCP, UDP, DHCP/BOOTP, and TCP retransmissions. Selected packet details were inspected in the Packet Details pane. The Protocol Hierarchy and Conversations windows were used to review traffic statistics. Captures and screenshots were saved as evidence.

Packet counts and byte totals in this report refer to the selected capture and the statistics view used at the time. They should not be treated as universal measurements of the network.

Practical 01: ICMP and DNS Traffic Analysis

Objective: To capture and examine ICMP and DNS packets using Wireshark.

Procedure: Wireshark was used to capture network traffic. The `ping -n 4 1.1.1.1` command generated ICMP traffic, and `nslookup example.com` generated DNS traffic. The captured traffic was examined using the `icmp` and `dns` display filters.

Observations:

- ICMP Echo Request and Echo Reply packets were visible.
- DNS Query and DNS Response packets were visible.
- The packet capture was saved as a `.pcapng` file.

Conclusion: The captured packets demonstrated ICMP request/reply communication and DNS query/response activity. The results show how Wireshark can be used to observe and analyze network protocols.

Practical 02: TCP Three-Way Handshake Analysis

Objective: To analyze the TCP three-way handshake using Wireshark.

Procedure: A TCP connection was initiated using the Windows PowerShell command `TestNetConnection example.com -Port 80`. The captured packets were filtered using `tcp.flags.syn == 1` and `tcp.stream == 10`.

Observations:

1. Packet 35 contained the SYN flag, indicating a connection request from the client.
2. Packet 37 contained SYN and ACK flags, indicating the server's response.
3. Packet 39 contained the ACK flag, completing the TCP three-way handshake.
4. Subsequent packets showed TLS handshake and application-data traffic over the established connection.

Conclusion: The three-way handshake was successfully identified in Wireshark. The captured packets demonstrated how TCP establishes a reliable connection before transferring application data.

Practical 03: DHCP Traffic Analysis

Objective: To capture and analyze DHCP traffic using Wireshark.

Procedure: A Wi-Fi network capture was started in Wireshark. DHCP traffic was generated by attempting to renew the Windows network configuration. The captured packets were examined using the `dhcp || bootp` display filter.

Observations:

- DHCP Request packets were observed.
- DHCP ACK packets were observed.
- The Request and ACK packet details were examined to identify the DHCP message types and transaction information.

Conclusion: The captured DHCP Request and ACK packets demonstrated the client-server exchange involved in DHCP configuration renewal. The complete Discover–Offer–Request–ACK sequence was not confirmed unless all four message types were observed.

Practical 04: TCP and UDP Traffic Analysis Using Wireshark

Objective :

To capture and analyze TCP and UDP packets using Wireshark, identify their source and destination ports, examine TCP flags, and understand UDP packet length.

Tools Used:

- Operating System: Windows 11
- Network analysis tool: Wireshark
- Network interface: Wi-Fi

Methodology :

1. Started a packet capture on the Wi-Fi network interface.
2. Generated network traffic.
3. Applied the tcp display filter to identify TCP packets.
4. Examined TCP source port, destination port, and flags.
5. Applied the udp display filter to identify UDP packets.
6. Examined UDP source port, destination port, and length.
7. Saved the capture for further analysis.

Observed Results:

TCP Packet

Field	Observed value
Source port	443
Destination port	50698
TCP flags	PSH, ACK

Analysis: Source port 443 is commonly used for HTTPS traffic. Destination port 50698 is an ephemeral client-side port in this connection. The PSH flag indicates that the sender is requesting that buffered data be passed to the receiving application promptly. The ACK flag indicates that the acknowledgment field is significant. These flags are consistent with data transfer in an established TCP connection.

UDP Packet

Field	Observed value
Source port	51957
Destination port	53
UDP length	46 bytes
Protocol column	UDP

Analysis: Source port 51957 is an ephemeral port, while destination port 53 is commonly associated with DNS services. This packet may be DNS-related traffic. However, since Wireshark displayed the protocol as UDP, the DNS interpretation should be confirmed by applying the dns display filter. The observed UDP length is 46 bytes, including the UDP header and payload.

Comparison of TCP and UDP:

TCP is connection-oriented and provides reliable, ordered data delivery using acknowledgments and retransmission mechanisms. UDP is connectionless and has a smaller header, without TCP-style delivery acknowledgments. UDP is commonly used by DNS and other applications that benefit from low overhead.

Conclusion:

The practical demonstrated how Wireshark can be used to inspect TCP and UDP traffic. The selected TCP packet contained source port 443, destination port 50698, and PSH and ACK flags. The selected UDP packet contained source port 51957, destination port 53, and a length of 46 bytes. These observations helped identify common port usage and understand the information available in packet headers.

The analysis is limited to the selected packets; it does not establish the behavior of every TCP or UDP packet in the capture.

Practical 05: TCP Retransmission Analysis

Objective: To identify TCP retransmission packets using Wireshark.

Filter used: tcp.analysis.retransmission

Observed results:

- The display filter returned 67 matching packets.
- A TCP stream with index 1 was identified during inspection.
- The port value recorded during inspection was 443.
- The value 50644 was recorded, but its corresponding field has not yet been confirmed.

Conclusion: Wireshark identified 67 packets matching the TCP retransmission filter in the selected capture. Stream 1 was identified for further inspection. The cause and impact of these retransmissions have not yet been established.

Practical 06: Protocol Hierarchy Analysis

Objective:

To examine TCP and UDP traffic statistics using Wireshark's Protocol Hierarchy feature.

Procedure:

The Protocol Hierarchy window was opened through `Statistics → Protocol Hierarchy`. The display filter was cleared, and the TCP and UDP packet counts and byte totals were recorded from the displayed statistics.

Observed Results:

Measurement	TCP	UDP
Packet count	2,461	248
Total bytes	51,628	1,984

The combined totals for the TCP and UDP rows were 2,709 packets and 53,612 bytes.

Analysis:

The observed statistics show that the selected capture contained 2,461 TCP packets and 248 UDP packets. The recorded TCP byte total was 51,628 bytes, while the UDP byte total was 1,984 bytes.

TCP therefore had a higher packet count and recorded byte total than UDP in this capture. These values describe the observed protocol statistics only. They do not identify the applications responsible for the traffic or explain why the protocol totals differ.

Conversation statistics were not recorded for this practical; therefore, no conclusions about individual TCP or UDP conversations are included.

Conclusion:

Wireshark's Protocol Hierarchy feature was used to summarize TCP and UDP traffic in the selected capture. The recorded results were 2,461 TCP packets containing 51,628 bytes and 248 UDP packets containing 1,984 bytes. This practical demonstrated how protocol statistics can be used to compare the observed packet counts and byte totals.

Overall Conclusion

This project provided practical experience in using Wireshark to inspect ICMP, DNS, TCP, UDP, and DHCP traffic. It also demonstrated TCP handshake analysis, retransmission filtering, and protocol statistics. The recorded observations show how packet-level inspection and traffic summaries can support basic network analysis. Conclusions were limited to the evidence collected during the practical activities.